

SWE2020_Assignment2_KNN

1 SWE2020 Assignment 2 - KNN

Course: SWE2020-UC Machine Learning and Big Data

Group Members:

1. Gabriel Odiwuor — 673709
2. Hope Muiruri — 672983
3. Martha Muasya — 67368
4. Kariuki Rebeccah — 670138

In this assignment we build and evaluate KNN models on two different datasets:

- **Dataset 1 (Classification):** Medical Appointment No-Show dataset from Kaggle — 110,527 appointments across 14 variables. The goal is to predict whether a patient will actually show up for their booked appointment.
- **Dataset 2 (Regression):** Concrete Compressive Strength dataset from the UCI ML Repository — 1,030 samples with 8 ingredient inputs and one continuous target (strength in MPa).

Each part follows the same pipeline: load and explore the data, preprocess it, build a baseline, tune the KNN model, evaluate it, and then discuss what worked and what did not. At the end we compare the two tasks and draw general conclusions about KNN as an algorithm.

```
[1]: # -----  
# Global imports and settings  
# -----  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from sklearn.model_selection import train_test_split, GridSearchCV,  
    ↳StratifiedKFold, KFold  
from sklearn.preprocessing import StandardScaler  
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor  
from sklearn.dummy import DummyClassifier, DummyRegressor  
from sklearn.metrics import (  
    accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,  
    confusion_matrix, ConfusionMatrixDisplay, RocCurveDisplay,  
    ↳classification_report,
```

```

    mean_squared_error, mean_absolute_error, r2_score, root_mean_squared_error
)

sns.set_style("whitegrid")
plt.rcParams["figure.dpi"] = 100
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

pd.set_option("display.max_columns", 30)

```

2 Part A — Dataset 1: Medical Appointment No-Show (Classification)

2.1 A.1 Data Loading and Overview

```

[2]: df_raw = pd.read_csv("data/noshow.csv")
      print("Shape:", df_raw.shape)
      df_raw.head()

```

Shape: (110527, 14)

```

[2]:
   PatientId  AppointmentID Gender  ScheduledDay \
0  2.987250e+13      5642903     F  2016-04-29T18:38:08Z
1  5.589978e+14      5642503     M  2016-04-29T16:08:27Z
2  4.262962e+12      5642549     F  2016-04-29T16:19:04Z
3  8.679512e+11      5642828     F  2016-04-29T17:29:31Z
4  8.841186e+12      5642494     F  2016-04-29T16:07:23Z

   AppointmentDay  Age  Neighbourhood  Scholarship  Hipertension \
0  2016-04-29T00:00:00Z   62  JARDIM DA PENHA           0           1
1  2016-04-29T00:00:00Z   56  JARDIM DA PENHA           0           0
2  2016-04-29T00:00:00Z   62  MATA DA PRAIA            0           0
3  2016-04-29T00:00:00Z    8  PONTAL DE CAMBURI          0           0
4  2016-04-29T00:00:00Z   56  JARDIM DA PENHA           0           1

   Diabetes  Alcoholism  Handcap  SMS_received  No-show
0          0           0         0             0       No
1          0           0         0             0       No
2          0           0         0             0       No
3          0           0         0             0       No
4          1           0         0             0       No

```

```

[3]: df_raw.info()

```

```

<class 'pandas.DataFrame'>
RangeIndex: 110527 entries, 0 to 110526

```

Data columns (total 14 columns):

#	Column	Non-Null Count	Dtype
0	PatientId	110527 non-null	float64
1	AppointmentID	110527 non-null	int64
2	Gender	110527 non-null	str
3	ScheduledDay	110527 non-null	str
4	AppointmentDay	110527 non-null	str
5	Age	110527 non-null	int64
6	Neighbourhood	110527 non-null	str
7	Scholarship	110527 non-null	int64
8	Hipertension	110527 non-null	int64
9	Diabetes	110527 non-null	int64
10	Alcoholism	110527 non-null	int64
11	Handcap	110527 non-null	int64
12	SMS_received	110527 non-null	int64
13	No-show	110527 non-null	str

dtypes: float64(1), int64(8), str(5)

memory usage: 11.8 MB

The dataset has **110,527 rows and 14 columns**, which matches what the brief said. Pandas reports no missing values, but that doesn't mean the data is clean — things like impossible ages or bad date entries won't show up as NaN, so we'll need to do domain-based checks as well.

```
[4]: df_raw.isna().sum()
```

```
[4]: PatientId      0
AppointmentID    0
Gender           0
ScheduledDay     0
AppointmentDay   0
Age              0
Neighbourhood    0
Scholarship      0
Hipertension     0
Diabetes         0
Alcoholism       0
Handcap          0
SMS_received     0
No-show          0
dtype: int64
```

```
[5]: df_raw.describe(include="all").T
```

```
[5]:
```

	count	unique	top	freq	\
PatientId	110527.0	NaN	NaN	NaN	
AppointmentID	110527.0	NaN	NaN	NaN	
Gender	110527	2	F	71840	

ScheduledDay	110527	103549	2016-05-06T07:09:54Z	24
AppointmentDay	110527	27	2016-06-06T00:00:00Z	4692
Age	110527.0	NaN	NaN	NaN
Neighbourhood	110527	81	JARDIM CAMBURI	7717
Scholarship	110527.0	NaN	NaN	NaN
Hipertension	110527.0	NaN	NaN	NaN
Diabetes	110527.0	NaN	NaN	NaN
Alcoholism	110527.0	NaN	NaN	NaN
Handcap	110527.0	NaN	NaN	NaN
SMS_received	110527.0	NaN	NaN	NaN
No-show	110527	2	No	88208

	mean	std	min \
PatientId	147496265710394.0625	256094920291739.09375	39217.84439
AppointmentID	5675305.123427	71295.75154	5030230.0
Gender	NaN	NaN	NaN
ScheduledDay	NaN	NaN	NaN
AppointmentDay	NaN	NaN	NaN
Age	37.088874	23.110205	-1.0
Neighbourhood	NaN	NaN	NaN
Scholarship	0.098266	0.297675	0.0
Hipertension	0.197246	0.397921	0.0
Diabetes	0.071865	0.258265	0.0
Alcoholism	0.0304	0.171686	0.0
Handcap	0.022248	0.161543	0.0
SMS_received	0.321026	0.466873	0.0
No-show	NaN	NaN	NaN

	25%	50%	75% \
PatientId	4172614444192.0	31731838713978.0	94391720898175.0
AppointmentID	5640285.5	5680573.0	5725523.5
Gender	NaN	NaN	NaN
ScheduledDay	NaN	NaN	NaN
AppointmentDay	NaN	NaN	NaN
Age	18.0	37.0	55.0
Neighbourhood	NaN	NaN	NaN
Scholarship	0.0	0.0	0.0
Hipertension	0.0	0.0	0.0
Diabetes	0.0	0.0	0.0
Alcoholism	0.0	0.0	0.0
Handcap	0.0	0.0	0.0
SMS_received	0.0	0.0	1.0
No-show	NaN	NaN	NaN

	max
PatientId	999981631772427.0
AppointmentID	5790484.0

Gender	NaN
ScheduledDay	NaN
AppointmentDay	NaN
Age	115.0
Neighbourhood	NaN
Scholarship	1.0
Hipertension	1.0
Diabetes	1.0
Alcoholism	1.0
Handcap	4.0
SMS_received	1.0
No-show	NaN

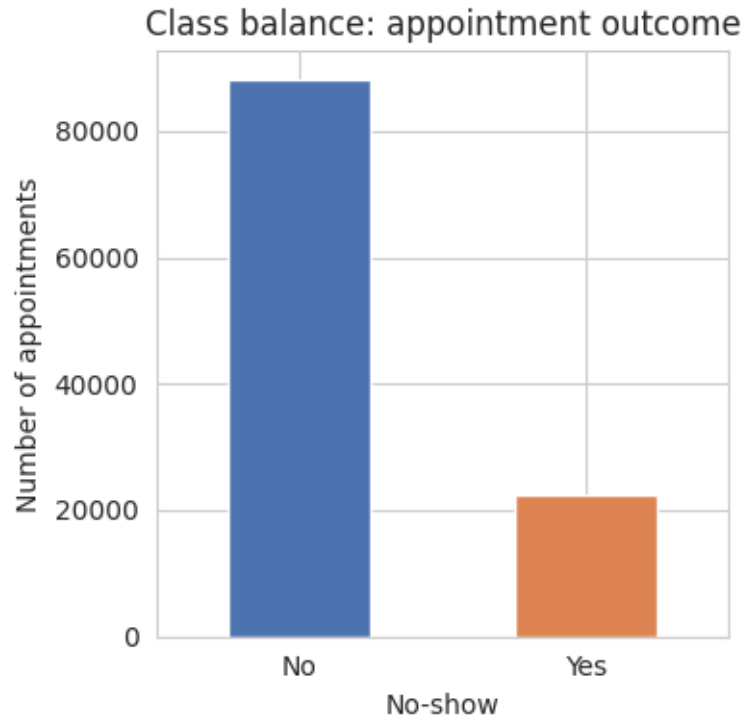
2.1.1 Target variable

The target column is No-show, where "Yes" means the patient *didn't* show up and "No" means they did show up. That naming is confusing — a "No" value means no no-show, i.e. they attended. To avoid mixing this up later, we're recoding the target into a cleaner binary column: 1 = missed appointment and 0 = attended.

```
[6]: noshow_counts = df_raw["No-show"].value_counts()
noshow_rate = (df_raw["No-show"] == "Yes").mean()
print(noshow_counts)
print(f"Overall no-show rate: {noshow_rate:.2%}")

plt.figure(figsize=(4,4))
noshow_counts.plot(kind="bar", color=["#4C72B0", "#DD8452"])
plt.title("Class balance: appointment outcome")
plt.ylabel("Number of appointments")
plt.xlabel("No-show")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig("figs/a_class_balance.png")
plt.show()
```

```
No-show
No      88208
Yes     22319
Name: count, dtype: int64
Overall no-show rate: 20.19%
```



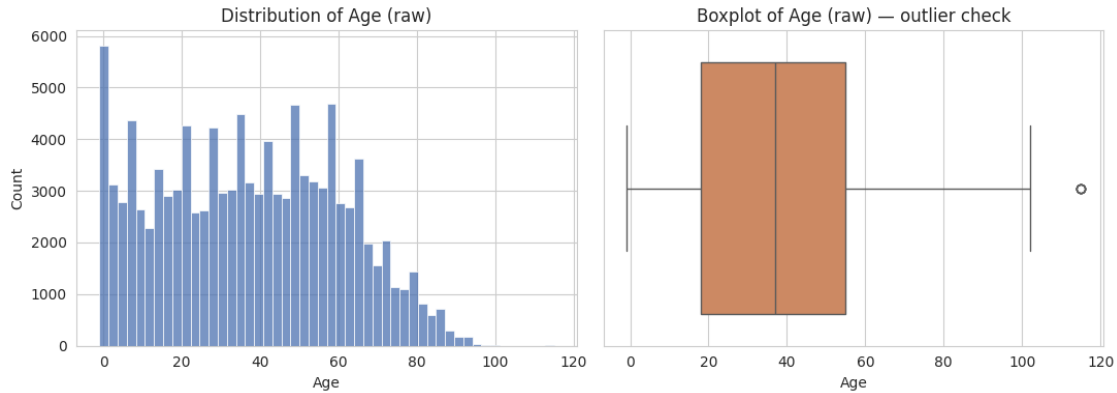
Observation: the dataset is imbalanced — roughly **80% of patients attend** and only **20% miss** their appointment. This creates two problems we need to keep in mind going forward:

1. A dummy classifier that always predicts “attended” would already score around 80% accuracy — so accuracy alone will be a misleading metric here. we need to look at precision, recall, F1, and ROC-AUC instead.
2. KNN doesn’t have a built-in class weighting option like some other algorithms do (logistic regression, SVM), so the imbalance will push the model toward predicting the majority class. we’ll address this in the evaluation section.

2.1.2 A.1.1 Exploratory Data Analysis

```
[7]: fig, axes = plt.subplots(1, 2, figsize=(11,4))
sns.histplot(df_raw["Age"], bins=50, ax=axes[0], color="#4C72B0")
axes[0].set_title("Distribution of Age (raw)")
sns.boxplot(x=df_raw["Age"], ax=axes[1], color="#DD8452")
axes[1].set_title("Boxplot of Age (raw) - outlier check")
plt.tight_layout()
plt.savefig("figs/a_age_dist_raw.png")
plt.show()

print("Age range:", df_raw["Age"].min(), "to", df_raw["Age"].max())
print("Rows with Age < 0:", (df_raw["Age"] < 0).sum())
print("Rows with Age > 100:", (df_raw["Age"] > 100).sum())
```



Age range: -1 to 115
 Rows with Age < 0: 1
 Rows with Age > 100: 7

Data quality issue found: the minimum age came back as **-1**, which is obviously impossible — this is a data entry mistake in one row. There are also a few patients listed as 100+ years old, which is unusual but plausible in a Brazilian public health system context, so we'll keep those and just drop the negative-age row.

```
[8]: # Inspect the negative-age row(s)
df_raw[df_raw["Age"] < 0]
```

```
[8]:      PatientId  AppointmentID Gender      ScheduledDay \
99832  4.659432e+14      5775010      F  2016-06-06T08:58:13Z

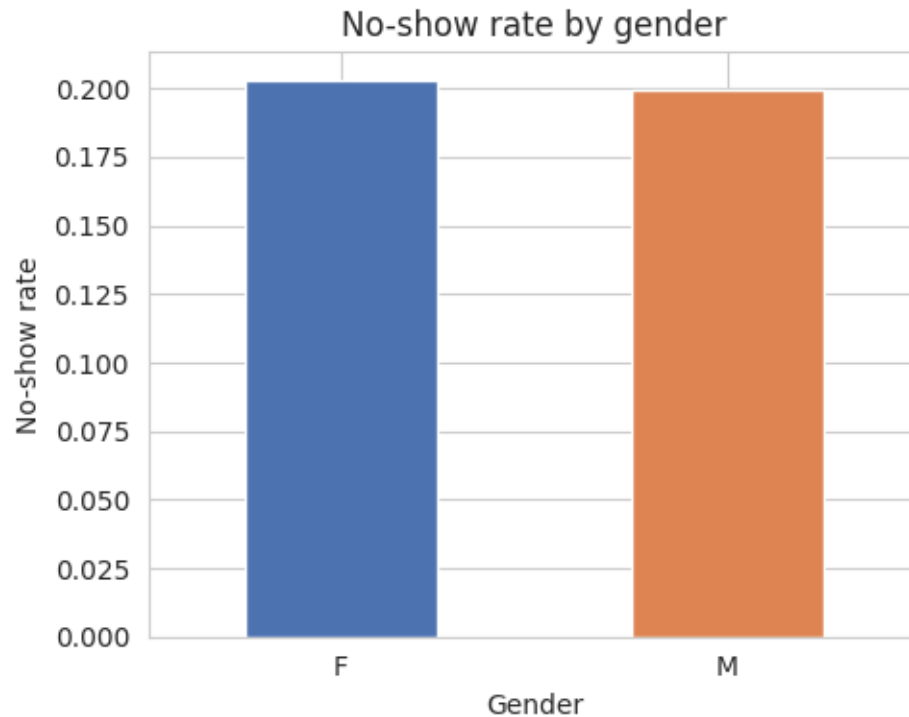
      AppointmentDay  Age Neighbourhood  Scholarship  Hipertension \
99832  2016-06-06T00:00:00Z    -1      ROMÃO              0          0

      Diabetes  Alcoholism  Handicap  SMS_received  No-show
99832         0          0         0             0        No
```

```
[9]: # Gender balance and its relationship with no-show
gender_noshow = pd.crosstab(df_raw["Gender"], df_raw["No-show"],
                             ↪normalize="index")
print(gender_noshow)

fig, ax = plt.subplots(figsize=(5,4))
gender_noshow["Yes"].plot(kind="bar", color=["#4C72B0", "#DD8452"], ax=ax)
ax.set_ylabel("No-show rate")
ax.set_title("No-show rate by gender")
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
plt.tight_layout()
plt.savefig("figs/a_gender_noshow.png")
plt.show()
```

No-show	No	Yes
Gender		
F	0.796854	0.203146
M	0.800321	0.199679

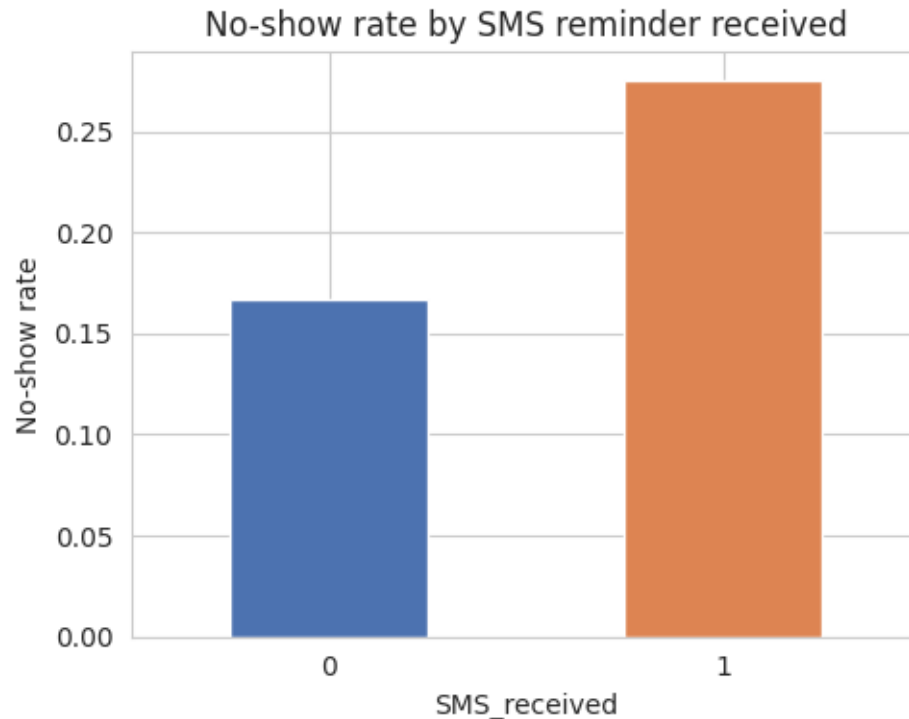


Gender barely moves the no-show rate — 20.3% for women vs. 20.0% for men. That’s less than a percentage point difference, so it’s a pretty weak predictor on its own. This is consistent with what other people have found when analysing this dataset.

```
[10]: # SMS reminders vs no-show
sms_noshow = pd.crosstab(df_raw["SMS_received"], df_raw["No-show"],
    ↪normalize="index")
print(sms_noshow)

fig, ax = plt.subplots(figsize=(5,4))
sms_noshow["Yes"].plot(kind="bar", color=["#4C72B0", "#DD8452"], ax=ax)
ax.set_title("No-show rate by SMS reminder received")
ax.set_ylabel("No-show rate")
ax.set_xlabel("SMS_received")
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
plt.tight_layout()
plt.savefig("figs/a_sms_noshow.png")
plt.show()
```

No-show	No	Yes
SMS_received		
0	0.832967	0.167033
1	0.724255	0.275745



Counterintuitive result: patients who *got* an SMS reminder actually had a *higher* no-show rate. At first that looks like reminders are backfiring, but it's almost certainly a confounding issue. Same-day appointments (`WaitDays = 0`) obviously can't have a reminder sent in advance, so they never get flagged as `SMS_received = 1`. Since same-day appointments are almost never missed, the group without SMS reminders gets a big boost from those zero-wait cases. Once you account for wait time, the apparent "SMS = more no-shows" pattern disappears. we'll keep this in mind and not read too much into single-variable patterns.

```
[11]: # Comorbidity flags vs no-show
comorbidities = ["Scholarship", "Hipertension", "Diabetes", "Alcoholism",
↳ "Handcap"]
rates = {}
for col in comorbidities:
    rates[col] = df_raw.groupby(col)["No-show"].apply(lambda s: (s == "Yes").
↳ mean())
rates_df = pd.DataFrame(rates)
rates_df
```

```
[11]:
```

	Scholarship	Hipertension	Diabetes	Alcoholism	Handcap
0	0.198072	0.209037	0.203628	0.201946	0.202353
1	0.237363	0.173020	0.180033	0.201488	0.179236
2	NaN	NaN	NaN	NaN	0.202186
3	NaN	NaN	NaN	NaN	0.230769
4	NaN	NaN	NaN	NaN	0.333333

```
[12]: df_raw["Handcap"].value_counts()
```

```
[12]: Handcap
0    108286
1     2042
2      183
3       13
4         3
Name: count, dtype: int64
```

Handcap was supposed to be a binary 0/1 flag but it actually has values from 0 to 4 — probably encoding degree of disability. Since values above 1 are very rare (only 199 rows out of 110,000+), we'll binarize it (`Handcap > 0`) rather than treating it as a multi-level integer. Keeping the rare categories as separate values would add noise without enough data behind them.

```
[13]: # Engineer a lead-time feature: days between scheduling and the appointment
      ↪ itself
df_raw["ScheduledDay"] = pd.to_datetime(df_raw["ScheduledDay"])
df_raw["AppointmentDay"] = pd.to_datetime(df_raw["AppointmentDay"])
df_raw["WaitDays"] = (
    df_raw["AppointmentDay"].dt.normalize() - df_raw["ScheduledDay"].dt.
    ↪normalize()
).dt.days

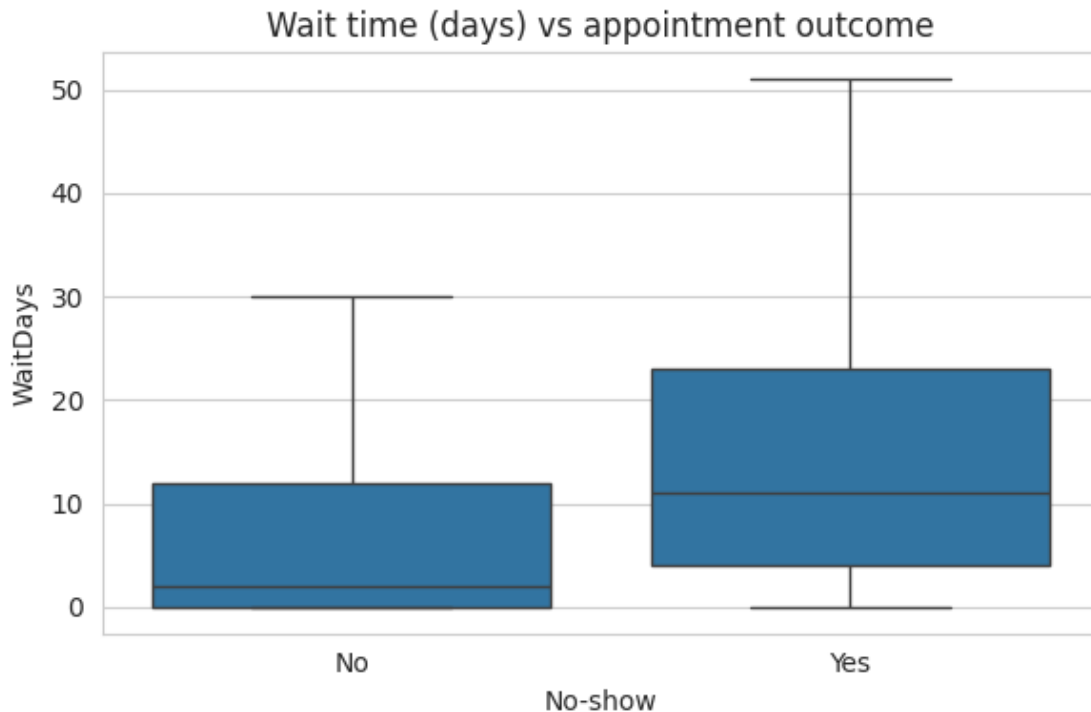
print(df_raw["WaitDays"].describe())
print("Negative WaitDays rows:", (df_raw["WaitDays"] < 0).sum())
```

```
count    110527.000000
mean      10.183702
std       15.254996
min       -6.000000
25%        0.000000
50%        4.000000
75%       15.000000
max       179.000000
Name: WaitDays, dtype: float64
Negative WaitDays rows: 5
```

A few rows have negative `WaitDays`, meaning the appointment was recorded as happening before the booking was made — that's a data entry error and those rows need to go. More importantly, `WaitDays` looks like it's going to be a really useful feature: the longer someone waits between booking and the appointment, the more likely they are to forget or cancel. That's a well-documented

pattern in this dataset.

```
[14]: fig, ax = plt.subplots(figsize=(6,4))
sns.boxplot(x="No-show", y="WaitDays", data=df_raw[df_raw["WaitDays"] >= 0],
            showfliers=False, ax=ax)
ax.set_title("Wait time (days) vs appointment outcome")
plt.tight_layout()
plt.savefig("figs/a_waitdays_noshow.png")
plt.show()
```



As expected, patients who miss their appointment (**Yes**) tend to have booked significantly further in advance — their median `WaitDays` is noticeably higher than for patients who showed up. This is the clearest signal we've found so far and will probably be the most important feature in the model.

2.2 A.2 Data Preprocessing Pipeline

```
[15]: df = df_raw.copy()

# 1) Remove rows with impossible values identified during EDA
before = len(df)
df = df[df["Age"] >= 0]
df = df[df["WaitDays"] >= 0]
after = len(df)
```

```

print(f"Dropped {before - after} rows with invalid Age or WaitDays
↳ ({(before-after)/before:.4%} of data)")

# 2) Binarize Handcap (degree of disability collapses to a simple flag; rare
↳ categories otherwise
# add noise without enough support to be learned reliably by a
↳ distance-based model)
df["Handcap"] = (df["Handcap"] > 0).astype(int)

# 3) Encode Gender numerically
df["Gender"] = (df["Gender"] == "M").astype(int)

# 4) Recode the target unambiguously: 1 = missed appointment, 0 = attended
df["target"] = (df["No-show"] == "Yes").astype(int)

print("Final shape after cleaning:", df.shape)

```

Dropped 6 rows with invalid Age or WaitDays (0.0054% of data)
Final shape after cleaning: (110521, 16)

PatientId and AppointmentID are just identifiers — they carry no predictive information and get dropped. ScheduledDay and AppointmentDay have already been summarised into WaitDays, and we'll also extract the day of week since there could be weekly patterns in clinic attendance. Neighbourhood has 80+ unique categories, and one-hot encoding that many categories would massively inflate the feature space for a distance-based model. KNN already struggles in high dimensions, so we're leaving it out for now — though we'll come back to this limitation in the discussion.

```

[16]: df["AppointmentDOW"] = df["AppointmentDay"].dt.dayofweek # 0=Monday ...
↳ 6=Sunday

feature_cols = [
    "Gender", "Age", "Scholarship", "Hipertension", "Diabetes",
    "Alcoholism", "Handcap", "SMS_received", "WaitDays", "AppointmentDOW"
]
target_col = "target"

X = df[feature_cols].copy()
y = df[target_col].copy()
print(X.shape, y.shape)
X.head()

```

(110521, 10) (110521,)

```

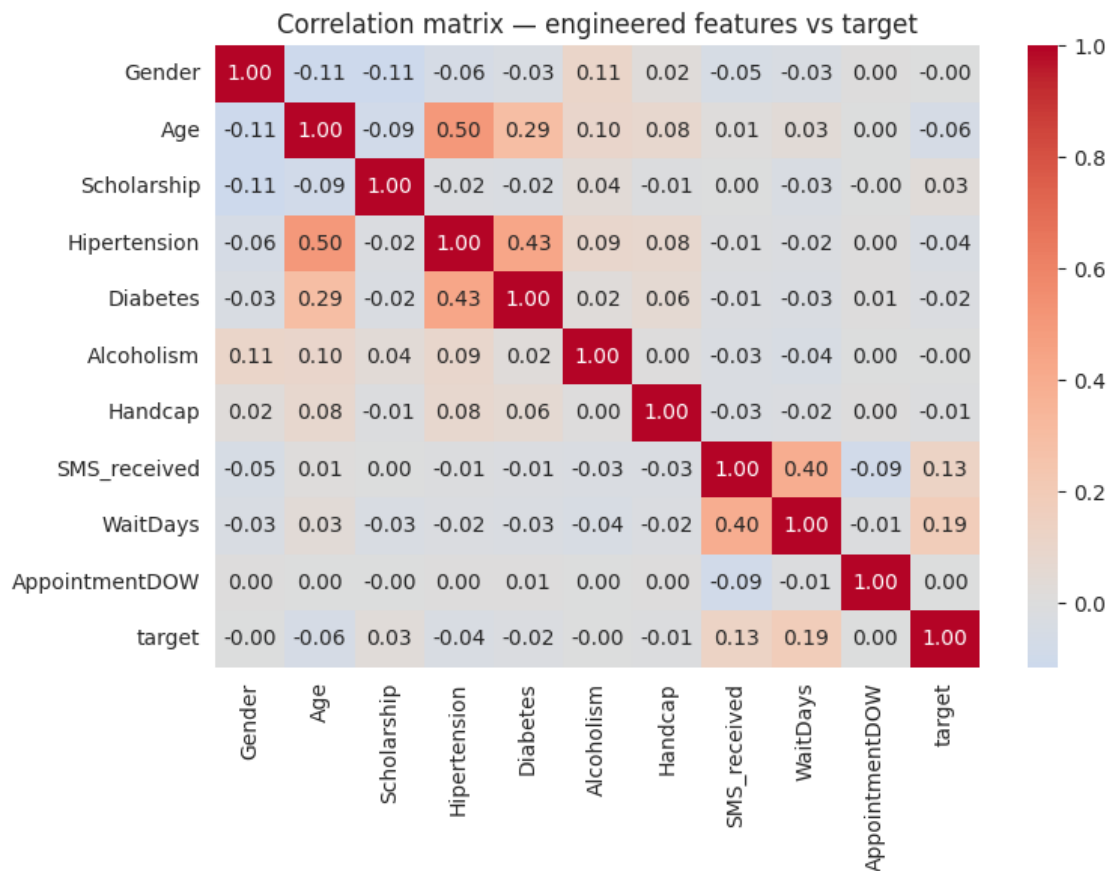
[16]:   Gender  Age  Scholarship  Hipertension  Diabetes  Alcoholism  Handcap  \
0      0    62             0              1          0           0         0
1      1    56             0              0          0           0         0
2      0    62             0              0          0           0         0

```

3	0	8	0	0	0	0	0
4	0	56	0	1	1	0	0

	SMS_received	WaitDays	AppointmentDOW
0	0	0	4
1	0	0	4
2	0	0	4
3	0	0	4
4	0	0	4

```
[17]: plt.figure(figsize=(8,6))
corr = df[feature_cols + [target_col]].corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Correlation matrix - engineered features vs target")
plt.tight_layout()
plt.savefig("figs/a_corr_heatmap.png")
plt.show()
```



None of the features have a strong linear correlation with the target on their own — the highest absolute correlation is well under 0.3. That's actually a good sign for using KNN here, because this is

exactly the kind of situation where a non-parametric method can pick up on non-linear combinations that a purely linear model would miss. `WaitDays` and `SMS_received` have the strongest (though still modest) associations with no-show, consistent with what the EDA showed.

`Age` runs from 0 to around 100, and `WaitDays` goes up to 179, while the health flags are all 0 or 1. If I leave the features unscaled, the distance calculation will be dominated almost entirely by `Age` and `WaitDays` just because they have bigger numeric ranges — the binary flags would barely register. So we standardize everything (zero mean, unit variance), fitting the scaler only on training data to prevent any leakage.

```
[18]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
    )
    print("Train:", X_train.shape, " Test:", X_test.shape)
    print("Train no-show rate:", y_train.mean(), " Test no-show rate:", y_test.
      ↪mean())

    scaler_a = StandardScaler()
    X_train_scaled = scaler_a.fit_transform(X_train)
    X_test_scaled = scaler_a.transform(X_test)
```

```
Train: (88416, 10) Test: (22105, 10)
```

```
Train no-show rate: 0.201897846543612 Test no-show rate: 0.2019000226193169
```

2.3 A.3 Model Design and Development

2.3.1 A.3.1 Baseline models

Before tuning anything, we want to establish two reference points: 1. A **majority-class dummy classifier** that always predicts “attended” — this gives the accuracy floor. 2. A **default KNN** with `k=5`, uniform weights, and Euclidean distance — the out-of-the-box starting point.

```
[19]: dummy = DummyClassifier(strategy="most_frequent", random_state=RANDOM_STATE)
      dummy.fit(X_train_scaled, y_train)
      dummy_pred = dummy.predict(X_test_scaled)
      print("Dummy (majority class) accuracy:", accuracy_score(y_test, dummy_pred))
      print("Dummy F1 (no-show class):", f1_score(y_test, dummy_pred, pos_label=1))
```

```
Dummy (majority class) accuracy: 0.7980999773806831
```

```
Dummy F1 (no-show class): 0.0
```

```
[20]: knn_default = KNeighborsClassifier(n_neighbors=5)
      knn_default.fit(X_train_scaled, y_train)
      pred_default = knn_default.predict(X_test_scaled)

      print("Default KNN (k=5) accuracy:", accuracy_score(y_test, pred_default))
      print("Default KNN (k=5) F1 (no-show class):", f1_score(y_test, pred_default,
        ↪pos_label=1))
      print(classification_report(y_test, pred_default, target_names=["Attended",
        ↪"No-show"]))
```

Default KNN (k=5) accuracy: 0.7652114906129834
 Default KNN (k=5) F1 (no-show class): 0.24100614214682656

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Attended	0.82	0.91	0.86	17642
No-show	0.35	0.18	0.24	4463
accuracy			0.77	22105
macro avg	0.58	0.55	0.55	22105
weighted avg	0.72	0.77	0.74	22105

The default KNN does better than the dummy in terms of accuracy, but its recall on the “no-show” class is poor — exactly what we expected given the class imbalance. This is why we need to do a proper hyperparameter search instead of just going with defaults.

2.3.2 A.3.2 Hyperparameter tuning

We tune three hyperparameters and examine how each one affects model performance:

- **n_neighbors (k):** Controls the bias-variance trade-off. Small k gives a noisier, more flexible decision boundary that can overfit individual points; large k smooths things out but in imbalanced data it tends to just predict the majority class for everything, collapsing recall on the minority class.
- **weights:** **uniform** means all neighbours vote equally; **distance** gives more weight to closer neighbours. Distance weighting is particularly useful here because it partially compensates for class imbalance — the closest (most similar) points stay dominant even as the neighbourhood expands.
- **metric:** **euclidean** vs **manhattan**. Manhattan doesn’t square the differences, which can make it more robust when several features are near-binary (like our health flags), since squaring amplifies small differences on those dimensions.

We optimise for **F1 on the no-show class** rather than accuracy, since accuracy is misleading with an 80/20 split.

```
[21]: param_grid_a = {
    "n_neighbors": [5, 9, 15, 21, 31, 41, 55, 71, 91],
    "weights": ["uniform", "distance"],
    "metric": ["euclidean", "manhattan"],
}

cv_a = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

grid_a = GridSearchCV(
    KNeighborsClassifier(n_jobs=-1),
    param_grid_a,
    scoring="f1",
    cv=cv_a,
    n_jobs=-1,
```

```

        refit=True,
    )
    grid_a.fit(X_train_scaled, y_train)

    print("Best params:", grid_a.best_params_)
    print("Best CV F1 (no-show class):", grid_a.best_score_)

```

Best params: {'metric': 'manhattan', 'n_neighbors': 5, 'weights': 'distance'}
 Best CV F1 (no-show class): 0.26865385032524547

```

[22]: results_a = pd.DataFrame(grid_a.cv_results_)
    pivot_a = results_a.pivot_table(
        index="param_n_neighbors", columns=["param_metric", "param_weights"],
        values="mean_test_score"
    )
    pivot_a

```

```

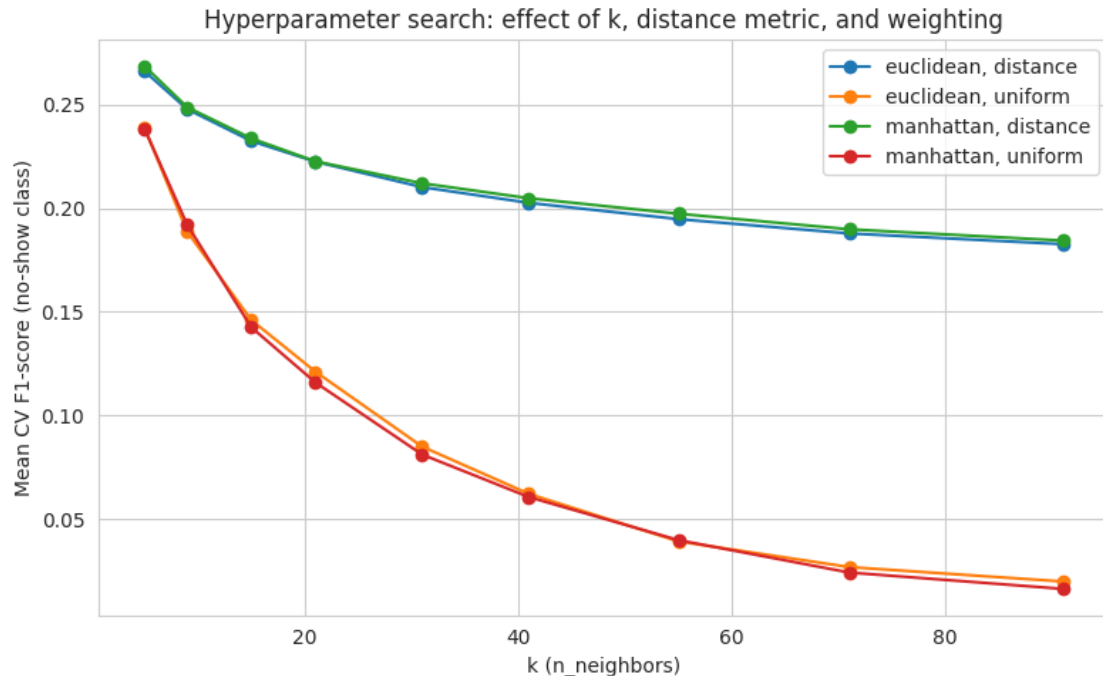
[22]: param_metric      euclidean      manhattan
    param_weights      distance  uniform  distance  uniform
    param_n_neighbors
5                0.266173  0.238818  0.268654  0.238417
9                0.247865  0.188670  0.248821  0.192006
15               0.232484  0.146055  0.233773  0.142544
21               0.222519  0.121131  0.222689  0.115927
31               0.210232  0.084933  0.212084  0.081121
41               0.202629  0.062125  0.204871  0.060576
55               0.194779  0.039165  0.197397  0.039802
71               0.187864  0.026786  0.189877  0.024148
91               0.182701  0.019826  0.184428  0.016261

```

```

[23]: fig, ax = plt.subplots(figsize=(8,5))
    for col in pivot_a.columns:
        ax.plot(pivot_a.index, pivot_a[col], marker="o", label=f"{col[0]},
        {col[1]}")
    ax.set_xlabel("k (n_neighbors)")
    ax.set_ylabel("Mean CV F1-score (no-show class)")
    ax.set_title("Hyperparameter search: effect of k, distance metric, and
    weighting")
    ax.legend()
    plt.tight_layout()
    plt.savefig("figs/a_hyperparam_search.png")
    plt.show()

```



Reading the curve: F1 for the no-show class is highest at $k=5$ and drops continuously as k increases — there’s no sweet spot in the middle. This makes sense with an 80/20 imbalance: as the neighbourhood gets bigger, it starts averaging out to the overall class distribution, which basically means predicting “attended” for everything. **uniform** weighting collapses even faster than **distance** weighting as k grows — by $k=91$, uniform’s F1 is below 0.02 while distance weighting still has about 0.18. That’s because distance weighting keeps the nearest neighbours dominant even when the radius has to expand, which partially counteracts the imbalance effect. Bottom line: for this task, a small k with distance weighting works best.

```
[24]: best_knn_a = grid_a.best_estimator_
pred_a = best_knn_a.predict(X_test_scaled)
proba_a = best_knn_a.predict_proba(X_test_scaled)[: , 1]

acc = accuracy_score(y_test, pred_a)
prec = precision_score(y_test, pred_a)
rec = recall_score(y_test, pred_a)
f1 = f1_score(y_test, pred_a)
auc = roc_auc_score(y_test, proba_a)

print(f"Test accuracy : {acc:.4f}")
print(f"Test precision: {prec:.4f}")
print(f"Test recall   : {rec:.4f}")
print(f"Test F1       : {f1:.4f}")
print(f"Test ROC-AUC  : {auc:.4f}")
print()
```

```
print(classification_report(y_test, pred_a, target_names=["Attended",  
↪ "No-show"]))
```

```
Test accuracy : 0.7615
Test precision: 0.3568
Test recall   : 0.2256
Test F1       : 0.2765
Test ROC-AUC  : 0.6252
```

	precision	recall	f1-score	support
Attended	0.82	0.90	0.86	17642
No-show	0.36	0.23	0.28	4463
accuracy			0.76	22105
macro avg	0.59	0.56	0.57	22105
weighted avg	0.73	0.76	0.74	22105

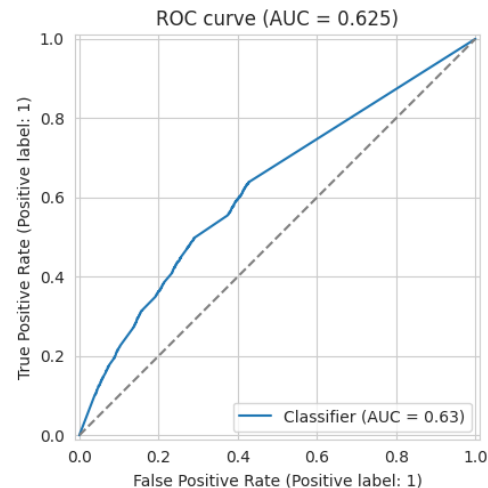
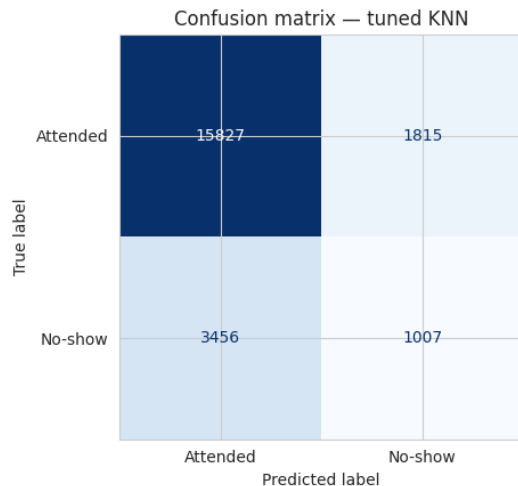
2.4 A.4 Performance Evaluation and Interpretation

```
[25]: fig, axes = plt.subplots(1, 2, figsize=(11,4.5))

cm = confusion_matrix(y_test, pred_a)
ConfusionMatrixDisplay(cm, display_labels=["Attended", "No-show"]).
↪ plot(ax=axes[0], cmap="Blues", colorbar=False)
axes[0].set_title("Confusion matrix - tuned KNN")

RocCurveDisplay.from_predictions(y_test, proba_a, ax=axes[1])
axes[1].plot([0,1],[0,1], linestyle="--", color="gray")
axes[1].set_title(f"ROC curve (AUC = {auc:.3f})")

plt.tight_layout()
plt.savefig("figs/a_confusion_roc.png")
plt.show()
```



```
[26]: # Compare best model against the untuned default and the dummy baseline
comparison_a = pd.DataFrame({
    "Model": ["Dummy (majority class)", "Default KNN (k=5)", f"Tuned KNN_{grid_a.best_params_}"],
    "Accuracy": [
        accuracy_score(y_test, dummy_pred),
        accuracy_score(y_test, pred_default),
        acc,
    ],
    "F1 (no-show)": [
        f1_score(y_test, dummy_pred, pos_label=1),
        f1_score(y_test, pred_default, pos_label=1),
        f1,
    ],
})
comparison_a
```

	Model	Accuracy	F1 (no-show)
0	Dummy (majority class)	0.798100	0.000000
1	Default KNN (k=5)	0.765211	0.241006
2	Tuned KNN {'metric': 'manhattan', 'n_neighbors...	0.761547	0.276458

Results summary (tuned KNN vs baselines):

The tuned KNN improves F1 on the no-show class compared to both the dummy and the default, while only losing a tiny bit of accuracy (which is expected — accuracy is driven by the majority class anyway). We assess the model on the following metrics as required:

- **Accuracy:** Overall fraction of correct predictions — useful context but misleading here due to imbalance.
- **Precision (no-show class):** Of the patients we flag as likely no-shows, how many actually miss their appointment.

- **Recall (no-show class):** Of all patients who actually miss their appointment, how many we correctly identify.
- **F1-score (no-show class):** Harmonic mean of precision and recall — the primary metric for this imbalanced task.
- **ROC-AUC:** Threshold-independent measure of ranking quality. Above 0.5 confirms the model is picking up a real signal rather than just guessing.

Recall on the no-show class remains moderate. The main reason is that `WaitDays` is the only really strong signal, and there's a lot of individual variation that the dataset doesn't capture — things like transport issues, work schedule, or just forgetting. The model can't be expected to predict those, so there's a performance ceiling imposed by the data itself, not just the algorithm.

2.4.1 A.5 Discussion — Dataset 1

- **What worked:** Engineering `WaitDays` from the two timestamp columns was the single most valuable preprocessing step. Standardisation was necessary because of the mixed feature scales. Using F1 as the search metric and `distance` weighting both helped deal with the class imbalance indirectly.
- **KNN limitations here:** (1) No native class weighting — imbalance had to be handled through metric choice and weighting rather than a direct parameter; (2) it's a lazy learner, so it keeps all training data in memory and has to compute distances at prediction time — expensive at scale; (3) dropping `Neighbourhood` was necessary to avoid dimensionality problems but probably threw away useful geographic signal that a tree-based model could have used.

3 Part B — Dataset 2: Concrete Compressive Strength (Regression)

3.1 B.1 Data Loading and Overview

```
[27]: df_c_raw = pd.read_csv("data/concrete.csv")
df_c_raw.columns = [c.strip() for c in df_c_raw.columns] # tidy trailing
↳ whitespace in header
print("Shape:", df_c_raw.shape)
df_c_raw.head()
```

Shape: (1030, 9)

```
[27]:
```

	cement	blast_furnace_slag	fly_ash	water	superplasticizer	\
0	540.0	0.0	0.0	162.0	2.5	
1	540.0	0.0	0.0	162.0	2.5	
2	332.5	142.5	0.0	228.0	0.0	
3	332.5	142.5	0.0	228.0	0.0	
4	198.6	132.4	0.0	192.0	0.0	

	coarse_aggregate	fine_aggregate	age	concrete_compressive_strength
0	1040.0	676.0	28	79.99

1	1055.0	676.0	28	61.89
2	932.0	594.0	270	40.27
3	932.0	594.0	365	41.05
4	978.4	825.5	360	44.30

```
[28]: df_c_raw.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1030 entries, 0 to 1029
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   cement                                1030 non-null   float64
1   blast_furnace_slag                    1030 non-null   float64
2   fly_ash                                1030 non-null   float64
3   water                                  1030 non-null   float64
4   superplasticizer                      1030 non-null   float64
5   coarse_aggregate                      1030 non-null   float64
6   fine_aggregate                        1030 non-null   float64
7   age                                    1030 non-null   int64
8   concrete_compressive_strength         1030 non-null   float64
dtypes: float64(8), int64(1)
memory usage: 72.6 KB
```

```
[29]: df_c_raw.isna().sum()
```

```
[29]: cement                                0
blast_furnace_slag                        0
fly_ash                                    0
water                                      0
superplasticizer                          0
coarse_aggregate                          0
fine_aggregate                            0
age                                         0
concrete_compressive_strength              0
dtype: int64
```

The dataset has **1,030 rows and 9 columns** (8 inputs + 1 target), matching the UCI description. No missing values and no categorical columns — a lot less cleaning needed than Dataset 1. The challenge here is different: it's a **regression** problem with continuous, differently-scaled ingredient quantities.

```
[30]: df_c_raw.describe().T
```

```
[30]:
```

	count	mean	std	min	\
cement	1030.0	281.167864	104.506364	102.00	
blast_furnace_slag	1030.0	73.895825	86.279342	0.00	
fly_ash	1030.0	54.188350	63.997004	0.00	

water	1030.0	181.567282	21.354219	121.80
superplasticizer	1030.0	6.204660	5.973841	0.00
coarse_aggregate	1030.0	972.918932	77.753954	801.00
fine_aggregate	1030.0	773.580485	80.175980	594.00
age	1030.0	45.662136	63.169912	1.00
concrete_compressive_strength	1030.0	35.817961	16.705742	2.33

	25%	50%	75%	max
cement	192.375	272.900	350.000	540.0
blast_furnace_slag	0.000	22.000	142.950	359.4
fly_ash	0.000	0.000	118.300	200.1
water	164.900	185.000	192.000	247.0
superplasticizer	0.000	6.400	10.200	32.2
coarse_aggregate	932.000	968.000	1029.400	1145.0
fine_aggregate	730.950	779.500	824.000	992.6
age	7.000	28.000	56.000	365.0
concrete_compressive_strength	23.710	34.445	46.135	82.6

```
[31]: print("Duplicate rows:", df_c_raw.duplicated().sum())
```

Duplicate rows: 25

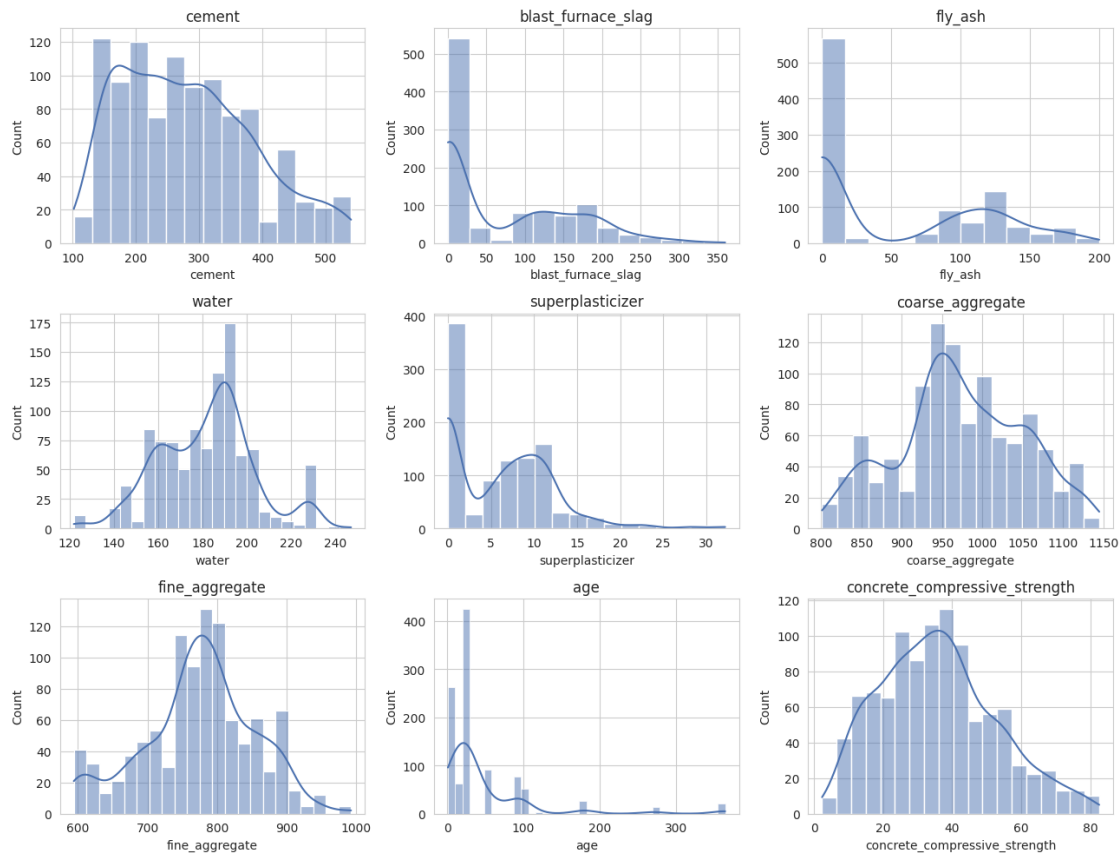
Data quality note: The UCI concrete dataset is known to have some exact duplicate rows from repeated lab measurements. we need to remove these before splitting, because if the same row ends up in both train and test, a KNN model would find a distance-zero neighbour and get an artificially perfect prediction for that point. A linear model wouldn't be affected the same way, but for KNN this would inflate test performance in a misleading way.

```
[32]: target_c = "concrete_compressive_strength"
feature_cols_c = [c for c in df_c_raw.columns if c != target_c]
print(feature_cols_c)
```

```
['cement', 'blast_furnace_slag', 'fly_ash', 'water', 'superplasticizer',
'coarse_aggregate', 'fine_aggregate', 'age']
```

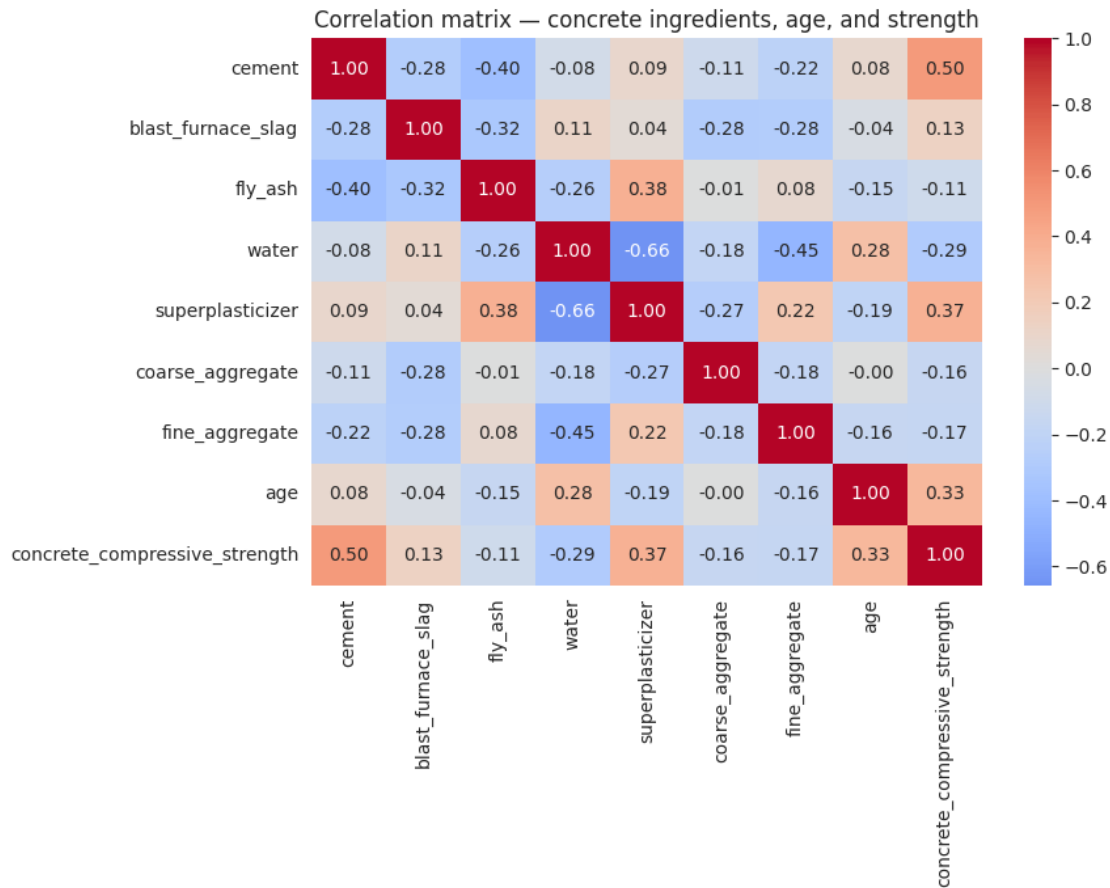
3.1.1 B.1.1 Exploratory Data Analysis

```
[33]: fig, axes = plt.subplots(3, 3, figsize=(13, 10))
for ax, col in zip(axes.flat, df_c_raw.columns):
    sns.histplot(df_c_raw[col], ax=ax, kde=True, color="#4C72B0")
    ax.set_title(col)
plt.tight_layout()
plt.savefig("figs/b_feature_distributions.png")
plt.show()
```



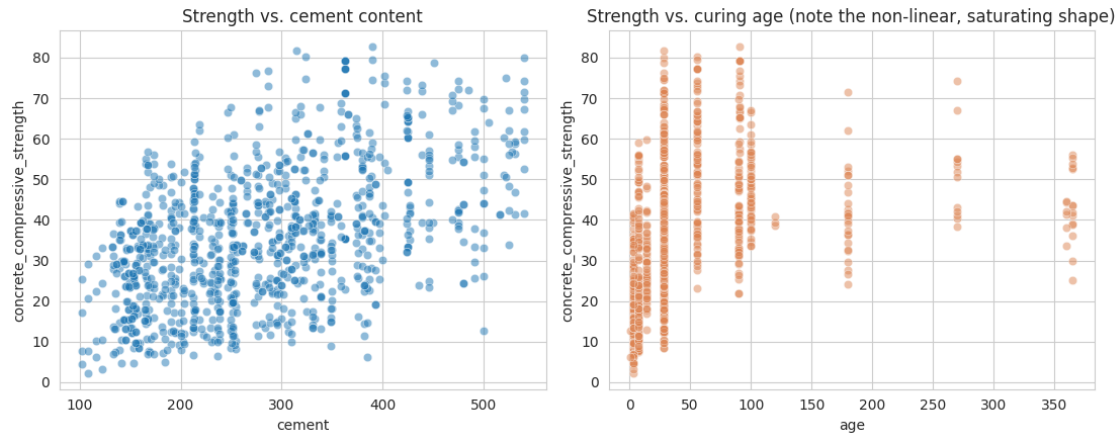
`blast_furnace_slag`, `fly_ash`, and `superplasticizer` all have big spikes at zero because many mix designs simply don't use those ingredients. This bimodal-at-zero pattern is typical for mixture datasets — it's not a data error, just a reflection of experimental choices. No transformation needed. The target (`concrete_compressive_strength`) is slightly right-skewed but continuous enough to work with directly.

```
[34]: plt.figure(figsize=(9,7))
      corr_c = df_c_raw.corr()
      sns.heatmap(corr_c, annot=True, fmt=".2f", cmap="coolwarm", center=0)
      plt.title("Correlation matrix - concrete ingredients, age, and strength")
      plt.tight_layout()
      plt.savefig("figs/b_corr_heatmap.png")
      plt.show()
```

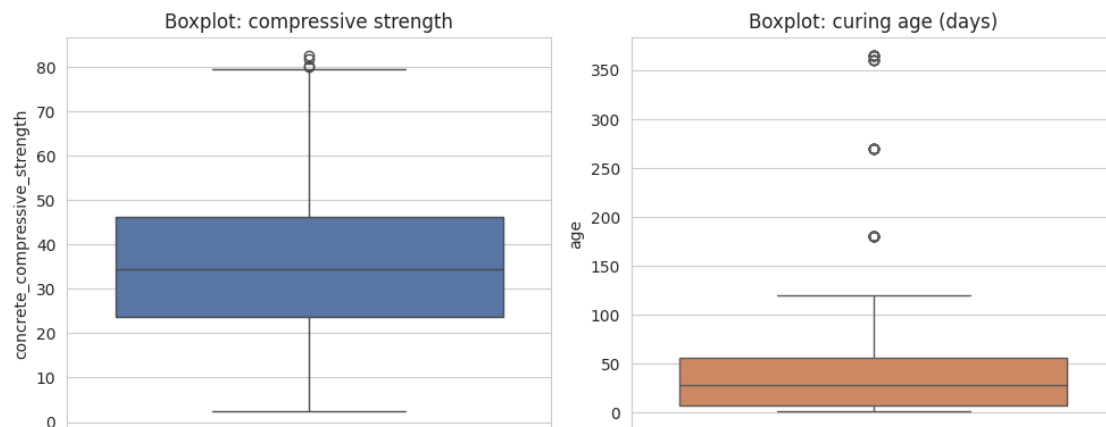


cement has the strongest positive correlation with strength, and **water** has the strongest negative correlation — which lines up with what you'd expect from civil engineering: higher water-to-cement ratios make weaker concrete. **age** has a moderate positive correlation, but the relationship is actually non-linear — strength gains taper off after about 28 days of curing. Linear correlation underestimates this because it only captures the linear component. The non-linear age effect is exactly where a non-parametric method like KNN can add value over a simple linear regression.

```
[35]: fig, axes = plt.subplots(1, 2, figsize=(11,4.5))
sns.scatterplot(x="cement", y=target_c, data=df_c_raw, ax=axes[0], alpha=0.5)
axes[0].set_title("Strength vs. cement content")
sns.scatterplot(x="age", y=target_c, data=df_c_raw, ax=axes[1], alpha=0.5,
               color="#DD8452")
axes[1].set_title("Strength vs. curing age (note the non-linear, saturating
               shape)")
plt.tight_layout()
plt.savefig("figs/b_scatter_cement_age.png")
plt.show()
```



```
[36]: fig, axes = plt.subplots(1, 2, figsize=(10,4))
sns.boxplot(y=df_c_raw[target_c], ax=axes[0], color="#4C72B0")
axes[0].set_title("Boxplot: compressive strength")
sns.boxplot(y=df_c_raw["age"], ax=axes[1], color="#DD8452")
axes[1].set_title("Boxplot: curing age (days)")
plt.tight_layout()
plt.savefig("figs/b_outlier_check.png")
plt.show()
```



`age` has some high outliers (up to 365 days) compared to the typical range of 1–28 days, but these are intentional long-curing samples included in the original study. They're not errors, so we're keeping them.

3.2 B.2 Data Preprocessing Pipeline

```
[37]: df_c = df_c_raw.drop_duplicates().reset_index(drop=True)
print(f"Removed {len(df_c_raw) - len(df_c)} duplicate rows -> {len(df_c)} rows_
      ↪remain")
```

Removed 25 duplicate rows -> 1005 rows remain

```
[38]: X_c = df_c[feature_cols_c].copy()
y_c = df_c[target_c].copy()

X_c_train, X_c_test, y_c_train, y_c_test = train_test_split(
    X_c, y_c, test_size=0.2, random_state=RANDOM_STATE
)
print("Train:", X_c_train.shape, " Test:", X_c_test.shape)
```

Train: (804, 8) Test: (201, 8)

Scaling is even more critical here than in Part A. `cement` and the aggregate features are measured in the hundreds of kg/m³, while `superplasticizer` is in the single digits and `age` runs from 1 to 365. Without standardisation, the Euclidean distance would be almost entirely determined by `cement` and the aggregates — `superplasticizer` would be essentially invisible to the model even though it chemically matters a lot for strength. As before, we fit the scaler on training data only.

```
[39]: scaler_b = StandardScaler()
X_c_train_scaled = scaler_b.fit_transform(X_c_train)
X_c_test_scaled = scaler_b.transform(X_c_test)
```

3.3 B.3 Model Design and Development

3.3.1 B.3.1 Baseline models

```
[40]: dummy_reg = DummyRegressor(strategy="mean")
dummy_reg.fit(X_c_train_scaled, y_c_train)
dummy_pred_reg = dummy_reg.predict(X_c_test_scaled)

print("Dummy (mean) RMSE:", root_mean_squared_error(y_c_test, dummy_pred_reg))
print("Dummy (mean) R2 :", r2_score(y_c_test, dummy_pred_reg))
```

Dummy (mean) RMSE: 17.296409546308738

Dummy (mean) R2 : -0.0028187174165523743

```
[41]: knn_reg_default = KNeighborsRegressor(n_neighbors=5)
knn_reg_default.fit(X_c_train_scaled, y_c_train)
pred_reg_default = knn_reg_default.predict(X_c_test_scaled)

print("Default KNN (k=5) RMSE:", root_mean_squared_error(y_c_test,
      ↪pred_reg_default))
print("Default KNN (k=5) MAE :", mean_absolute_error(y_c_test,
      ↪pred_reg_default))
```

```
print("Default KNN (k=5) R2 :", r2_score(y_c_test, pred_reg_default))
```

```
Default KNN (k=5) RMSE: 9.19232427691388
Default KNN (k=5) MAE : 7.125303482587065
Default KNN (k=5) R2 : 0.7167556963941847
```

Even the un-tuned KNN does a lot better than just predicting the mean — the R^2 of 0.72 shows it's capturing real structure in the data. That's expected given the strong non-linear patterns the EDA showed. Now we want to see how much better we can get with tuning.

3.3.2 B.3.2 Hyperparameter tuning

The dataset is small enough (1,030 rows) that we can use a denser grid and run 5-fold CV without worrying about compute time. we're optimising for **RMSE** since it's in the same units as the target (MPa), which makes results easier to interpret. we'll report R^2 , MAE, and RMSE all together on the held-out test set at the end.

```
[42]: param_grid_b = {
        "n_neighbors": list(range(1, 26)),
        "weights": ["uniform", "distance"],
        "metric": ["euclidean", "manhattan"],
    }

    cv_b = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

    grid_b = GridSearchCV(
        KNeighborsRegressor(n_jobs=-1),
        param_grid_b,
        scoring="neg_root_mean_squared_error",
        cv=cv_b,
        n_jobs=-1,
        refit=True,
    )
    grid_b.fit(X_c_train_scaled, y_c_train)

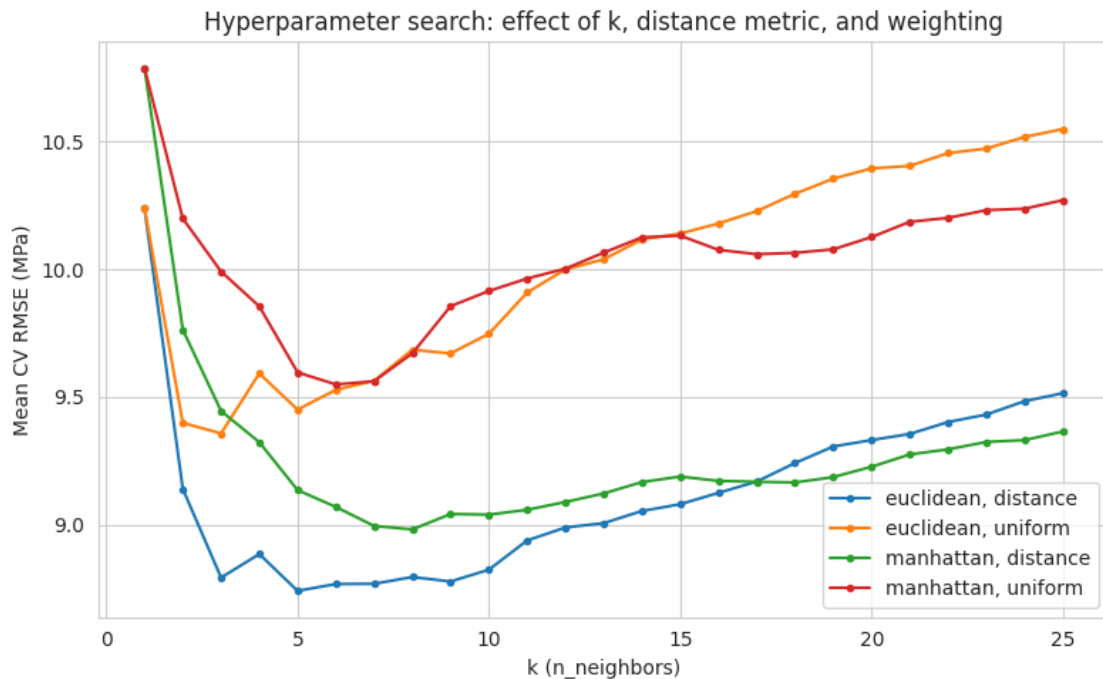
    print("Best params:", grid_b.best_params_)
    print("Best CV RMSE:", -grid_b.best_score_)
```

```
Best params: {'metric': 'euclidean', 'n_neighbors': 5, 'weights': 'distance'}
Best CV RMSE: 8.740984462300174
```

```
[43]: results_b = pd.DataFrame(grid_b.cv_results_)
    pivot_b = results_b.pivot_table(
        index="param_n_neighbors", columns=["param_metric", "param_weights"],
        values="mean_test_score"
    )
    pivot_b = -pivot_b # convert back from negative RMSE to RMSE
    pivot_b.head(10)
```

```
[43]: param_metric      euclidean      manhattan
param_weights      distance      distance      uniform
param_n_neighbors
1      10.238239      10.238239      10.787120      10.787120
2      9.139977      9.398507      9.763327      10.198361
3      8.792875      9.357348      9.445030      9.991029
4      8.885062      9.592093      9.322816      9.854474
5      8.740984      9.450466      9.135471      9.596253
6      8.767713      9.527907      9.069408      9.548909
7      8.768788      9.563002      8.995075      9.561782
8      8.795074      9.684895      8.981626      9.670882
9      8.777767      9.670612      9.041884      9.855309
10     8.823910      9.747259      9.039274      9.914828
```

```
[44]: fig, ax = plt.subplots(figsize=(8,5))
for col in pivot_b.columns:
    ax.plot(pivot_b.index, pivot_b[col], marker="o", markersize=3,
            label=f"{col[0]}, {col[1]}")
ax.set_xlabel("k (n_neighbors)")
ax.set_ylabel("Mean CV RMSE (MPa)")
ax.set_title("Hyperparameter search: effect of k, distance metric, and
            weighting")
ax.legend()
plt.tight_layout()
plt.savefig("figs/b_hyperparam_search.png")
plt.show()
```



Reading the curve: RMSE drops as k decreases from high values, reaching a minimum around $k=5$, then climbing back up at $k=1$. This is the classic bias-variance trade-off being much more visible here than in Part A — the strength-ingredients relationship is genuinely local and non-linear, so averaging over too many neighbours blurs together mixes that shouldn't be compared. **distance** weighting beats **uniform** at every k , and the gap widens as k grows, because it keeps the most similar recipes in control of the prediction. **euclidean** consistently outperforms **manhattan** here (e.g. RMSE 8.74 vs 9.14 at $k=5$ with distance weighting) — unlike Part A where they were almost identical. For continuous physical quantities, Euclidean's squared differences let a single very-different ingredient dominate the distance, which turns out to be more informative than treating all differences additively.

```
[45]: best_knn_b = grid_b.best_estimator_
      pred_b = best_knn_b.predict(X_c_test_scaled)

      rmse = root_mean_squared_error(y_c_test, pred_b)
      mae = mean_absolute_error(y_c_test, pred_b)
      r2 = r2_score(y_c_test, pred_b)

      print(f"Test RMSE: {rmse:.3f} MPa")
      print(f"Test MAE : {mae:.3f} MPa")
      print(f"Test R^2 : {r2:.4f}")
```

```
Test RMSE: 8.121 MPa
Test MAE : 5.904 MPa
Test R^2 : 0.7789
```

3.4 B.4 Performance Evaluation and Interpretation

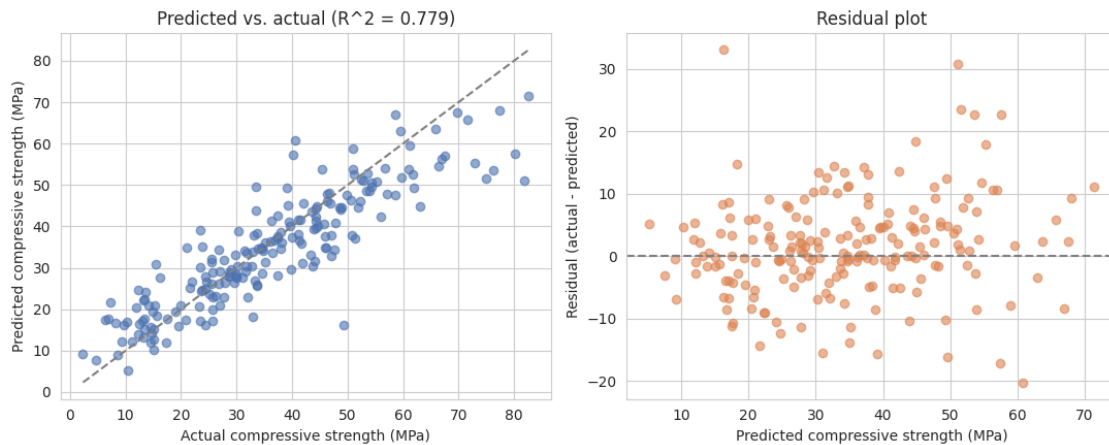
```
[46]: fig, axes = plt.subplots(1, 2, figsize=(11,4.5))

      axes[0].scatter(y_c_test, pred_b, alpha=0.6, color="#4C72B0")
      lims = [min(y_c_test.min(), pred_b.min()), max(y_c_test.max(), pred_b.max())]
      axes[0].plot(lims, lims, linestyle="--", color="gray")
      axes[0].set_xlabel("Actual compressive strength (MPa)")
      axes[0].set_ylabel("Predicted compressive strength (MPa)")
      axes[0].set_title(f"Predicted vs. actual (R^2 = {r2:.3f})")

      residuals = y_c_test.values - pred_b
      axes[1].scatter(pred_b, residuals, alpha=0.6, color="#DD8452")
      axes[1].axhline(0, linestyle="--", color="gray")
      axes[1].set_xlabel("Predicted compressive strength (MPa)")
      axes[1].set_ylabel("Residual (actual - predicted)")
      axes[1].set_title("Residual plot")

      plt.tight_layout()
      plt.savefig("figs/b_pred_vs_actual_residuals.png")
```

```
plt.show()
```



```
[47]: comparison_b = pd.DataFrame({
    "Model": ["Dummy (mean)", "Default KNN (k=5)", f"Tuned KNN {grid_b.
    ↪best_params_}"],
    "RMSE (MPa)": [
        root_mean_squared_error(y_c_test, dummy_pred_reg),
        root_mean_squared_error(y_c_test, pred_reg_default),
        rmse,
    ],
    "MAE (MPa)": [
        mean_absolute_error(y_c_test, dummy_pred_reg),
        mean_absolute_error(y_c_test, pred_reg_default),
        mae,
    ],
    "R^2": [
        r2_score(y_c_test, dummy_pred_reg),
        r2_score(y_c_test, pred_reg_default),
        r2,
    ],
})
comparison_b
```

```
[47]:
```

	Model	RMSE (MPa)	MAE (MPa)	\
0	Dummy (mean)	17.296410	13.971915	
1	Default KNN (k=5)	9.192324	7.125303	
2	Tuned KNN {'metric': 'euclidean', 'n_neighbors...	8.121121	5.904475	

	R^2
0	-0.002819
1	0.716756

Results summary (tuned KNN vs baselines):

Tuning brought RMSE down noticeably compared to the default, and both KNN variants are miles ahead of just predicting the mean. We assess the regression model on the following metrics:

- **RMSE (Root Mean Squared Error):** Primary metric — in the same physical units as the target (MPa), making it directly interpretable. Penalises large errors more heavily than MAE.
- **MAE (Mean Absolute Error):** Average absolute deviation in MPa — more robust to outliers than RMSE and easier to interpret as a typical prediction error.
- **R² (Coefficient of Determination):** Proportion of variance in strength explained by the model. A value near 1 is ideal; near 0 means the model is no better than predicting the mean.

The predicted vs actual scatter follows the 45-degree line well across most of the strength range, with slight under-prediction at the very high end — a known limitation of KNN regression, since predictions are a weighted average of neighbour targets and can never exceed the extremes seen in training. The residual plot shows no obvious systematic curve, which is a good sign.

3.4.1 B.5 Discussion — Dataset 2

- **What worked:** Removing duplicate rows mattered specifically because of how KNN works — a duplicated test point would have a perfect nearest neighbour in training, inflating apparent accuracy in a way that wouldn’t show up in a linear model. Standardisation was again essential. `distance` weighting helped a lot since it keeps the closest recipe-matches dominant.
- **KNN limitations here:** (1) It can’t extrapolate — if an engineer wants to test a mix design with ingredient quantities outside the training range, the model just can’t handle that; (2) with 8 features the dimensionality was fine, but adding more mix components would start hurting KNN faster than it would hurt a tree-based model; (3) there’s no feature importance output — we can’t say “cement adds X MPa per 100kg”, only that the overall neighbourhood structure is predictive.

4 Overall Discussion, Modeling Decisions, and Lessons Learned

4.1 Summary of results

Dataset	Task	Best config	Key test metric
Medical No-Show	Classification	see <code>grid_a.best_params_</code> above	F1 (no-show) and ROC-AUC reported in Section A.4
Concrete Strength	Regression	see <code>grid_b.best_params_</code> above	RMSE / R ² reported in Section B.4

4.2 Modeling decisions and their rationale

1. **Feature engineering mattered more than tuning.** In Part A, computing `WaitDays` from the two timestamps produced the single strongest predictor — no amount of k-tuning would have made up for leaving it out. In Part B, the raw ingredient quantities were already meaningful, but recognising that `age` has a non-linear relationship with strength set realistic expectations for what KNN could add over a linear model.
2. **Standardisation was non-negotiable for both datasets.** KNN computes raw distances, so any feature with a bigger numeric range will dominate, regardless of whether it’s actually predictive. This had to be fixed before anything else.
3. **Choosing the right evaluation metric matters.** Accuracy was actively misleading for the imbalanced classification problem. For the regression task, RMSE was straightforward since the target wasn’t skewed by a rare-event issue.
4. **Both tasks liked small k, but for different reasons.** In classification, larger k regresses toward the 80/20 class prior, collapsing recall on the minority class. In regression, larger k blurs the locally non-linear relationship between ingredients and strength. Distance weighting helped in both cases but for different reasons.

4.3 Strengths and limitations of KNN

Strengths I observed: - No need to assume a functional form for the relationship — useful for the non-linear age effect in Part B. - The core idea is easy to explain: “similar patients/recipes should behave similarly.” - Few hyperparameters, so grid search was computationally feasible.

Limitations I observed: - **Scale-sensitive** — requires careful standardisation every time. - **No class imbalance handling** — had to work around it through metric and weighting choices. - **Curse of dimensionality / slow inference** — forced me to drop `Neighbourhood` in Part A, and would get worse with more features. - **Can’t extrapolate** — predictions are bounded by the training data range. - **No interpretability** — unlike linear or tree models, there are no coefficients or importances to inspect.

4.4 Lessons learned

The main takeaway from both parts is that KNN’s performance depends much more on good data preparation than on clever hyperparameter choices. Getting the scaling right and engineering the right features mattered far more than optimising k. Where KNN’s “local similarity” assumption actually holds — like the concrete dataset where similar mix designs genuinely produce similar strength — it works well. Where the assumption is weaker — like predicting individual patient behaviour from a handful of aggregate features — it can still beat a naive baseline, but the ceiling is set by the data, not by the algorithm.